

- **What is a process?**
 - A running instance of a program.
- **What is the difference between a program and a process?**
 - Program = stored code on disk.
 - Process = program currently running in memory.
- **What does the ps command show?**
 - Lists running processes.
- **What do the columns PID, TTY, TIME, and CMD mean?**
 - PID = Process ID
 - TTY = terminal associated with process
 - TIME = CPU time used
 - CMD = command/program name
- **What additional information does ps -f show?**
 - More detailed info like parent PID, user, and start time.
- **What does the & symbol do at the end of a command?**
 - Runs the process in the background.
- **What does \$! store?**
 - PID of the most recent background process.
- **Why is the process ID important?**
 - Used to manage or terminate processes.
- **What did pgrep -a sleep show?**
 - Displayed running sleep processes with their commands.
- **What does the wait command do?**
 - Waits for background jobs to finish.
- **What does exit status 0 usually mean?**
 - Command succeeded.
- **What does a non-zero exit status usually mean?**
 - Command failed or had an error.
- **Why would someone run multiple processes in the background?**
 - To multitask and save time.
- **Why is wait useful when working with background jobs?**
 - Ensures tasks finish before continuing.
- **What order did the jobs finish in?**
 - Usually shortest/fastest job first.
- **What is a signal?**
 - A message sent to a process to control it.
- **What is the purpose of the kill command?**
 - Sends signals to processes.
- **What is the difference between sending a signal and deleting a file?**
 - Signals affect running processes; deleting removes files from storage.

- **What are SIGTERM, SIGKILL, and SIGINT commonly used for?**
 - SIGTERM = graceful stop
 - SIGKILL = force stop
 - SIGINT = interrupt from keyboard (Ctrl+C)
- **What happened after you sent SIGTERM?**
 - The process attempted to close cleanly.
- **What happened after you sent SIGKILL?**
 - The process stopped immediately.
- **Why is SIGKILL stronger than SIGTERM?**
 - It cannot be ignored by the process.
- **Why should you usually try SIGTERM before SIGKILL?**
 - Allows cleanup and safe shutdown.
- **What does ps -C sleep do?**
 - Shows processes named sleep.
- **What does pgrep do?**
 - Find process IDs by name.
- **Why might pgrep be easier than manually searching through all processes?**
 - Automatically filters matching processes.
- **What does top show that ps does not show as clearly?**
 - Real-time process activity.
- **What do %CPU and %MEM mean?**
 - CPU and memory usage percentages.
- **Why is top useful for system monitoring?**
 - Shows live system performance.
- **What does the nice command do?**
 - Starts a process with a chosen priority.
- **What does the renice command do?**
 - Changes priority of a running process.
- **What does the NI column represent?**
 - Nice value (priority level).
- **Does a higher nice value mean higher priority or lower priority?**
 - Lower priority.
- **What is an environment variable?**
 - A variable shared with processes and shells.
- **What does \$HOME store?**
 - User's home directory.
- **What does \$USER store?**
 - Current username.
- **What does \$SHELL store?**
 - Current shell path.

- **What does \$PATH store?**
 - Directories searched for commands.
- **What is the difference between a local shell variable and an environment variable?**
 - Local = only current shell.
 - Environment = inherited by child processes.
- **What does export do?**
 - Make a variable an environment variable.
- **Which variable was visible inside the child shell?**
 - Exported variable.
- **Which variable was not visible inside the child shell?**
 - Local shell variable.
- **Why did ./scripts/hello_path.sh work?**
 - A full/relative path was provided.
- **Why did hello_path.sh only work after changing PATH?**
 - Script directory was added to PATH.
- **Why is PATH important when running commands?**
 - Let the shell locate executables.
- **What is setuid?**
 - Runs file with owner's permissions.
- **What is setgid?**
 - Runs with group permissions/shared group ownership.
- **What is the sticky bit?**
 - Restricts file deletion in shared directories.
- **Where did you see the sticky bit in the output?**
 - As t in permissions (example: /tmp).
- **Why is the sticky bit commonly used on /tmp?**
 - Prevents users from deleting others' files.
- **What does sort do by default?**
 - Sorts lines alphabetically ascending.
- **What does sort -r do?**
 - Reverse sort order.
- **What does sort -u do?**
 - Sorts and removes duplicates.
- **Why did we use tail -n +2 when sorting the scores file?**
 - Skipped the header line.
- **Did sort change the original file?**
 - No, unless redirected to overwrite.
- **What does grep do?**
 - Searches text for patterns.
- **What does grep -i do?**

- Case-insensitive search.
- **What does `grep -n` do?**
 - Shows line numbers.
- **What does `grep -v` do?**
 - Shows non-matching lines.
- **What does `grep -c` do?**
 - Counts matching lines.
- **What does `grep -R` do?**
 - Recursive search through directories.
- **What does `awk` do?**
 - Processes and analyzes text by fields.
- **What does `-F '\t'` mean?**
 - Uses tab as field separator.
- **What does `$1` mean in `awk`?**
 - First field.
- **What does `$0` mean in `awk`?**
 - Entire line.
- **What does `NR` mean?**
 - Current record/line number.
- **What does `NF` mean?**
 - Number of fields in current line.
- **How did the average-score command work?**
 - Summed scores and divided by number of records.
- **What does `head` show by default?**
 - First 10 lines.
- **What does `tail` show by default?**
 - Last 10 lines.
- **What does `head -n 5` do?**
 - Shows first 5 lines.
- **What does `tail -n 5` do?**
 - Shows the last 5 lines.
- **What does `tail -n +3` do?**
 - Starts output from line 3 onward.
- **How did the command using both `head` and `tail` print only lines 4 through 8?**
 - `head` limited lines, `tail` selected subset.
- **What is the file approach?**
 - Save intermediate output to a file.
- **What is the pipe approach?**
 - Send output directly between commands.
- **Which approach avoids creating a temporary file?**

- Pipe approach.
- **Why are pipes useful in Linux?**
 - Connect small commands efficiently.
- **What does tee do?**
 - Displays output and writes to a file.
- **How is tee different from regular output redirection using >?**
 - tee shows output on screen too.
- **What does tee -a do?**
 - Appends instead of overwriting.
- **What does cut do?**
 - Extracts columns or characters.
- **What does cut -f1 do?**
 - Shows first field.
- **What does cut -f1,3 do?**
 - Shows first and third fields.
- **What does cut -c1-10 do?**
 - Shows characters 1–10.
- **What does -d ' ' mean?**
 - Uses space as a delimiter.
- **What does paste do?**
 - Merges lines side by side.
- **What delimiter does paste use by default?**
 - Tab.
- **What does paste -d ',' do?**
 - Uses comma delimiter.
- **What does paste -s do?**
 - Combines lines serially into one line.
- **What does uniq do?**
 - Removes adjacent duplicate lines.
- **Why did we sort the file before using uniq?**
 - Groups duplicate together.
- **What does uniq -c show?**
 - Counts duplicate occurrences.
- **What does uniq -d show?**
 - Shows only duplicates.
- **What does uniq -u show?**
 - Shows only unique lines.
- **Why are duplicate lines easier to remove after sorting?**
 - Sorting places duplicates next to each other.
- **What does wc show by default?**

- Lines, words, and bytes.
- **What does wc -l count?**
 - Lines.
- **What does wc -w count?**
 - Words.
- **What does wc -c count?**
 - Bytes/characters.
- **Why is wc useful in pipelines?**
 - Quickly counts processed output.
- **What did the grep part of the pipeline do?**
 - Filtered matching lines.
- **What did the cut part of the pipeline do?**
 - Extracted fields/columns.
- **What did the sort part of the pipeline do?**
 - Ordered the data.
- **What did the uniq -c part of the pipeline do?**
 - Counted duplicates.
- **What did the tee part of the pipeline do?**
 - Saved and displayed output.
- **Why is this an example of using small Linux tools together?**
 - Each command performs one task in sequence.
- **What did both commands do?**
 - Counted lines in a file.
- **Which command uses an extra cat process?**
 - cat file | wc -l
- **Why is wc -l < file usually more direct than cat file | wc -l?**
 - Avoids unnecessary process creation.
- **Why do people still sometimes use cat file | command when building pipelines?**
 - Easier to read or extend.
- **What does the script do?**
 - Automates a sequence of commands.
- **Why did we use variables named input and output?**
 - Makes script reusable and readable.
- **Why did we make the script executable with chmod +x?**
 - Allows direct execution.
- **Why is a script useful when you want to repeat the same pipeline later?**
 - Saves time and reduces errors.